Contents

1 day 7: apis II **1**
1.1 review: API requests, methods, calls (20m) 1
1.1.1 what is this code doing, line by line? 2
1.1.2 Review: into dataframes 2
1.1.3 Review: KEY error: 3
1.2 NYT API! 3
1.3 sign up for an NYTimes API account & key 3
1.4 authentication & pagination: (40m) 3
1.4.1 Creating API calls with keys 3
1.4.2 How do we understand our data structure? 4
1.4.3 Paginating through results 5
1.5 BREAK 6
1.6 group challenge: the NYPL API 6

1 day 7: apis II

Goals:

- review MET API,
  - how to request, parse, and extract data
  - start to do a little data analysis of results
- examine concepts: authentication & pagination
  - NYT API
  - self guided exploration of NYPL API
- discuss article, start to brainstrom what we want to do

1.1 review: API requests, methods, calls (20m)

- discuss the anatomy of an API request
  - root, path, endpoint
  - making the call
- interpreting some pre-written code
- discussing common error: KEY error

1.1.1 what is this code doing, line by line?

```
import requests

base_url = "https://collectionapi.metmuseum.org"
path = "/public/collection/v1.1/search"
query = "?q=woman"

url = f'{base_url}{path}{query}'
r = requests.get(url)
parsed = r.json()

for item in parsed['objectIDs'][:10]:
    base_url = 'https://collectionapi.metmuseum.org'
    path = '/public/collection/v1/objects/'
    url = f'{base_url}{path}{item}'
    response = requests.get(url)
    parsed = response.json()
    first_ten.append(parsed)
```

1.1.2 Review: into dataframes

Put it into a dataframe. Make a couple of charts - pie chart, bar chart, just for demonstration

```
    # let's get a bunch of this data into lists
import pandas as pd

titles = []
names = []
genders = []
depts = []
countries = []
urls = []

for item in first_ten:
    title = item.get('artistGender')
    titles.append(title)
    name = item.get('artistDisplayName')
    names.append(name)
```

```
    gender = item.get('artistGender')
    genders.append(gender)
    dept = item.get('department')
    depts.append(dept)
    country = item.get('country')
    countries.append(country)
    url = item.get('objectURL')
    urls.append(url)

df = pd.DataFrame({
  'title': titles,
  'name': names,
  'gender': genders,
  'department': depts,
  'country': countries,
  'link': urls
})
```

1.1.3 Review: KEY error:

```
for item in first_fifty:
  if item.get('artistDisplayName'):
    print(item['artistDisplayName'])

for item in first_fiftu:
  title = item.get('artistDisplayName')
  print(title)
```

1.2 NYT API!

1.3 sign up for an NYTimes API account & key

1.4 authentication & pagination: (40m)

1.4.1 Creating API calls with keys

- requesting an API key
- adding the key

```
# key
key = 'HHhPAjnfEF2EmLtf9PdGsLEQZizR3iQu'
```

```
# query
query = 'migrant'

# base URL
url = f'https://api.nytimes.com/svc/search/v2/articlesearch.json?&q={query}&api-key={ke

response = requests.get(url)

response # 200

type(response) # Response
```

1.4.2 How do we understand our data structure?

- we poke at the response
- parse our data
- we find different structure of data
  - keep going deeper into the data using keys()

```
parsed = response.json()

parsed.keys() # dict_keys(['status', 'copyright', 'response'])

parsed['response'].keys() # dict_keys(['docs', 'meta'])

type(parsed['response']['metadata']) # dict

parsed['response']['metadata']

type(parsed['response']['docs']) # list

# The very first item in ‘docs‘ appears to be information about a
# single article.

parsed['response']['docs'][0]
```

```
parsed['response']['docs'][0].keys()
```

If I wanted to proceed to get this data, I would write a loop.

1.4.3 Paginating through results

- We have 10 articles, because the NYTimes API only allows us to get 10 results at a time.
- We can paginate through the results, by adding:
  - `&page=` parameter,
  - `sleep()` function,
    - which allows us to insert pauses in our request (as to not overload the NYTimes servers)
- write a loop that goes through a `range()` of 5 pages, and does a search for each page.
- append the results for each page to a list called results
- check out our results, where is our data?

```
from time import sleep
results = []
query = 'migrant'

for i in range(0, 5):
    url = f'https://api.nytimes.com/svc/search/v2/articlesearch.json?q={query}&page={i}
    response = requests.get(url)
    parsed = response.json()
    articles = parsed['response']['docs']
    results.append(articles)
    sleep(6) # sleep at least 6 seconds not to overload the servers
```

In the end, we have a list of results, 5 lists to be exact (each list represents one page of articles). Each result (or page) contains information about 10 articles that matched our search.

```
results[0][0].keys() # for the very first item on the very first page.

results[0][0]['abstract']
```

1.5 BREAK

1.6 group challenge: the NYPL API

Walk them through the authentication process.

Challenge: use the search path, and figure out a way to get more information about the individual objects in the search path results.